STRUCTURED OUTPUT

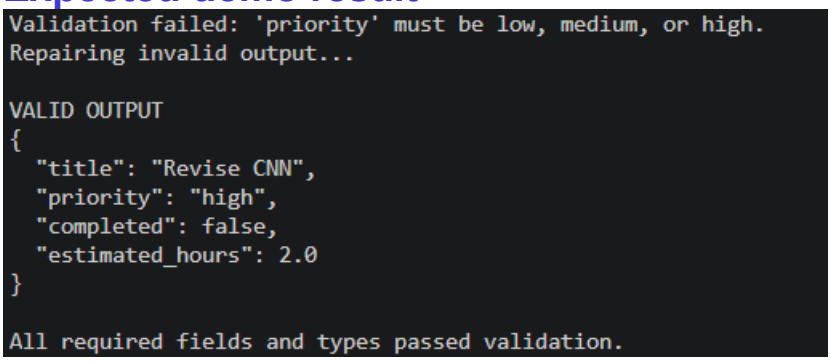
Task 02 — JSON Schema, Validation, and Retry/Repair

Objective: Build a small Python script that receives model-generated JSON, checks that it matches a schema, rejects invalid values, and attempts repair before failing safely.

Requirements covered: required schema, strict type checks, validation errors, retry/repair path, and predictable typed output.

Dependencies: Python standard library only. No API key or external package is required for the included demo.

Expected demo result



The demo intentionally starts with invalid model output and then supplies corrected JSON through the repair callback.

1. Schema

The script accepts exactly four fields:

Field	Type	Rule
title	string	Must not be empty
priority	string	Must be low, medium, or high
completed	boolean	Must be true or false, not a string
estimated_hours	number	Must be between 0 and 100

2. Validation strategy

The program first parses the response using json.loads(). It then verifies that the result is a JSON object, that all required fields exist, that no extra fields are present, and that each value has the correct type and allowed range. Invalid output raises OutputValidationError instead of being passed to the application.

3. Retry and repair path

get_valid_output() makes an initial model call. If parsing or validation fails, it passes the bad output and the validation error to repair_call(). It retries up to the configured limit. If the output remains invalid, it raises a clear error rather than returning unsafe data.

4. How to run

1. Download structured_output_validator.py.
2. Open a terminal in the downloaded file's folder.
3. Run: **python structured_output_validator.py**
4. Confirm that the output shows a validation failure, a repair attempt, and then valid JSON.

5. Connecting a real model

For a live model, replace model_call() with the chosen model/API request. Replace repair_call() with a second request that includes the validation error and invalid output, instructs the model to return only corrected JSON, and then let the same validator check the repaired response. Do not trust repaired output until validation succeeds.

6. Conclusion

This implementation meets the task's core goal: the application receives only parsed, validated, typed data. It rejects malformed JSON, missing or extra fields, incorrect types, invalid priority values, and out-of-range estimates. The retry/repair path handles recoverable failures and stops safely after the retry limit.

Submission note: The included run is a local demonstration using simulated responses. If the internship brief requires screenshots from a live model or a particular API, add those separately and follow the provider's exact submission instructions.

Appendix — Complete Python code

```
"""
Task 02: Structured Output
A small Python script that parses model JSON, validates its schema and types,
and retries through a repair callback when the output is invalid.

Run:
    python structured_output_validator.py

This demo uses simulated model responses, so it runs without an API key or
third-party packages. Replace model_call() with your model/API call when needed.
"""
import json
from typing import Any, Callable, Dict


class OutputValidationError(ValueError):
    """Raised when model output is invalid or does not match the schema."""


def validate_result(data: Any) -> Dict[str, Any]:
    """Validate the required schema and return typed data."""
    if not isinstance(data, dict):
        raise OutputValidationError("Output must be a JSON object.")

    required = {"title", "priority", "completed", "estimated_hours"}
    missing = required - data.keys()
    extra = data.keys() - required
    if missing:
        raise OutputValidationError(f"Missing fields: {sorted(missing)}")
    if extra:
        raise OutputValidationError(f"Unexpected fields: {sorted(extra)}")

    # Exact type checks matter: in Python, bool is a subclass of int.
    if not isinstance(data["title"], str) or not data["title"].strip():
        raise OutputValidationError("'title' must be a non-empty string.")

    if not isinstance(data["priority"], str) or data["priority"] not in {
        "low", "medium", "high"
    }:
        raise OutputValidationError("'priority' must be low, medium, or high.")

    if type(data["completed"]) is not bool:
        raise OutputValidationError("'completed' must be a boolean.")

    if type(data["estimated_hours"]) not in (int, float):
        raise OutputValidationError("'estimated_hours' must be a number.")
    if data["estimated_hours"] < 0 or data["estimated_hours"] > 100:
        raise OutputValidationError("'estimated_hours' must be between 0 and 100.")

    # Return a fresh object with predictable types and field order.
    return {
        "title": data["title"].strip(),
        "priority": data["priority"],
        "completed": data["completed"],
        "estimated_hours": float(data["estimated_hours"]),
    }


def parse_and_validate(raw_text: str) -> Dict[str, Any]:
    """Parse JSON text and validate its structure and values."""
    try:
        data = json.loads(raw_text)
    except json.JSONDecodeError as exc:
        raise OutputValidationError(f"Invalid JSON: {exc.msg}") from exc
    return validate_result(data)


def get_valid_output(
    model_call: Callable[[], str],
    repair_call: Callable[[str, str], str],
    max_retries: int = 2,
) -> Dict[str, Any]:
    """
    Try the model, validate its response, then ask a repair function to fix it.
    max_retries is the number of repair attempts after the first response.
    """
    raw = model_call()
    for attempt in range(max_retries + 1):
        try:
            return parse_and_validate(raw)
        except OutputValidationError as exc:
            if attempt == max_retries:
                raise OutputValidationError(
                    f"Output remained invalid after {max_retries} repair attempt(s): {exc}"
                ) from exc
            raw = repair_call(raw, str(exc))
    raise OutputValidationError("Unexpected validation failure.")
```

```
# ---- Demo callbacks: no API key needed ----
responses = iter([
    '{"title": "Revise CNN", "priority": "urgent", "completed": "no", "estimated_hours": -2}',
    '{"title": "Revise CNN", "priority": "high", "completed": false, "estimated_hours": 2}'
])


def model_call() -> str:
    """Demo model: first returns invalid data, then a corrected response."""
    return next(responses)


def repair_call(bad_output: str, error_message: str) -> str:
    """Demo repair: in a real app, send both items to the model and request JSON only."""
    print("Validation failed:", error_message)
    print("Repairing invalid output...")
    return next(responses)


if __name__ == "__main__":
    try:
        result = get_valid_output(model_call, repair_call, max_retries=2)
        print("\nVALID OUTPUT")
        print(json.dumps(result, indent=2))
        print("\nAll required fields and types passed validation.")
    except OutputValidationError as exc:
        print("ERROR:", exc)
```